

Integrating rsonpath with PostgreSQL: A Research Report

Dominik Strak

Krzysztof Lembryk

May 2026

Abstract

This report investigates the feasibility and performance of integrating the SIMD-accelerated JSONPath engine, rsonpath, with PostgreSQL through a Rust extension built using pgrx. We implemented several extension variants and a simple GIN operator class that enables index-assisted filtering of candidate rows. The resulting system was evaluated against PostgreSQL’s native JSONPath implementation on datasets ranging from single JSON documents of up to 90 MB to multi-gigabyte JSONL collections containing millions of records.

Our experiments show that rsonpath substantially outperforms PostgreSQL’s native JSONPath on large documents, achieving speedups of up to 745x for highly selective path queries while also handling JSON inputs significantly larger than those accepted by PostgreSQL’s `jsonb` representation. However, this advantage decreases with workloads consisting of many small documents, where PostgreSQL’s built-in implementation benefits from lower per-row overhead and direct access to the `jsonb` format. Profiling reveals that PostgreSQL infrastructure costs, particularly data decompression and data movement, dominate execution time when rsonpath is used through an extension, which is especially evident when working with millions of small files.

These results demonstrate that the rsonpath extension can provide dramatic benefits for large-document workloads in PostgreSQL, whereas for multi-gigabyte JSONL collections with smaller files, some work still needs to be done to achieve competitive performance compared to PostgreSQL’s native JSONPath.

1 Introduction

JSON has become a standard format for representing semi-structured data, and modern relational databases support it natively. PostgreSQL provides two JSON column types: `json`, which stores the raw text, and `jsonb`, which stores a parsed binary representation amenable to indexing and direct field access. Since version 12, PostgreSQL also supports the SQL/JSON path language for querying within `jsonb` values via `jsonb_path_query` and related functions. As JSON storage in PostgreSQL grows in practical use, the cost of querying it becomes a relevant performance concern.

Independently of PostgreSQL, the rsonpath project [1] has demonstrated that JSONPath queries can be evaluated much faster using SIMD instructions, by skipping large portions of the document that cannot contain a match. This raises a natural question: can PostgreSQL benefit from rsonpath’s architecture, and what does it take to bring the two together?

This report describes our attempt to answer that question. We build a PostgreSQL extension using pgrx [2], expose rsonpath through it, and compare it against PostgreSQL’s native JSONPath.

2 Motivation

JSON is now a routine part of relational schema design. Applications store event payloads, configuration blobs, document metadata, and semi-structured records as JSON columns in PostgreSQL. As these columns grow, so does the cost of querying inside them, and any speedup in JSONPath evaluation has direct practical value for read-heavy workloads.

Rsonpath operates as a single-pass streaming engine over raw JSON text. It uses SIMD to skip irrelevant fragments of the document, so the work it does scales with output size rather than input size. This is most useful where PostgreSQL is weakest: large documents with selective queries.

Our objective is to bridge the two systems and measure the results. Concretely, this raises three engineering questions:

- How is a JSON value handed off from PostgreSQL’s storage layer to a Rust extension function, and what does that cost?
- How much of the observed runtime is in PostgreSQL infrastructure (e.g., TOAST decompression, which restores compressed or out-of-line values before access, tuple construction, function dispatch) versus in rsonpath itself?
- Can a custom index filter candidate rows cheaply enough to make rsonpath competitive on many-row workloads, where per-call overhead dominates?

3 Design

3.1 pgrx

The extension is built using pgrx [2], a Rust framework for PostgreSQL extensions. It provides macros that translate Rust function signatures into PostgreSQL function declarations, wraps the PostgreSQL C ABI in safe Rust abstractions, and handles SQL schema generation.

We created three extensions:

1. rsonpath_ext_json - extension that returns result as a table with json column
2. rsonpath_ext_str - extension that returns result as a table with string column
3. rsonpath_ext_count - extension that returns count

3.2 The @@@ operator

The extension defines a binary infix operator `jsonb @@@ query_text` that returns `true` when the rsonpath query has at least one match in the document. This lets queries be written as `WHERE data @@@ '$.x.y'` rather than as positional function calls, which is both more natural in SQL and required to enable index support.

3.3 GIN operator class

To accelerate selective queries over large collections of documents, the extension defines a GIN (Generalized Inverted Index) operator class. GIN is a PostgreSQL access method designed for composite values that decompose into sets of element-level keys. Internally, GIN maintains for each key a posting list – the list of row IDs where that key appears – analogous to the index at the back of a textbook. Built-in uses include indexing the elements of arrays, the lexemes of full-text vectors, and the keys or path components of `jsonb` columns.

Plugging rsonpath into GIN requires three things from us.

A boolean operator on the indexed type. PostgreSQL’s planner can only attach an index to operator applications, not to function calls; this is why @@@ exists. The operator returns `true` iff the rsonpath query has at least one match in the document.

A rule for decomposing a row into keys. We need to tell GIN what “keys” a `jsonb` document contributes to the index. We choose every object key name that appears anywhere in the document, found by a recursive walk. So the document `{"title": ..., "authors": [{"name": ...}]}` contributes the keys `{title, authors, name}`.

A rule for decomposing a query into required keys. For any incoming JSONPath, we need to tell GIN which keys a matching document must contain. We parse the JSONPath, find every `Name` selector (object-key navigation), and return the resulting set. For `$.records[*].name` that is `{records, name}`. For queries with no extractable keys (e.g. `$.*`), we fall back to `GIN_SEARCH_MODE_ALL`, which forces a sequential candidate scan.

These three rules are wired into PostgreSQL’s GIN support function interface: `extractValue` runs once per row at index build and on every INSERT or UPDATE; `extractQuery` runs once per scan; and `consistent` runs once per candidate row.

For a query of the form

```
SELECT * FROM t WHERE data @@@ '$.records[*].name';
```

two execution plans are possible.

Without GIN. The operator function is invoked once per row. For each row the document is passed to `rsonpath`, the JSONPath is parsed and compiled into an automaton, and the document is scanned.

With GIN. The planner uses a Bitmap Index Scan followed by a Bitmap Heap Scan. A Bitmap Index Scan does not fetch table rows directly; instead, it consults the GIN index and constructs a bitmap of row IDs that may satisfy the query. A Bitmap Heap Scan then uses this bitmap to retrieve the corresponding table rows and perform any required rechecks.

The execution proceeds as follows:

1. `extractQuery` runs once, returning the required keys.
2. GIN walks the posting lists for those keys and produces a stream of candidate row IDs – a row enters the stream as soon as it appears in at least one of the posting lists (i.e. contains at least one of the required keys). This is only the inclusion threshold, not the final filter.
3. For each candidate, GIN builds an array of booleans – one per required key, indicating whether that key is present in the row – and passes it to `consistent`. We return `true` only if all required keys are present; otherwise the row is dropped.
4. The Bitmap Heap Scan fetches each surviving row and re-invokes `@@@`, which runs the full `rsonpath` engine. The recheck is necessary because GIN only knows which keys appear in a row, not where in the tree they sit; only `rsonpath` can confirm the keys appear along the path the query requires.

We confirmed this call sequence empirically by adding log statements to each support function and inspecting the resulting log output during a controlled query.

4 Experimental setup

Hardware. Asus ZenBook S14 UX5406, Intel Core Ultra 7 258V (8 logical CPUs), 32 GB RAM, Linux Mint 22.3 with kernel 6.17.0.

Software. PostgreSQL 13.23 (compiled with gcc 13.3.0 on Ubuntu 24.04, instance managed by pgrx). PostgreSQL was rebuilt without `--enable-cassert` and without `RANDOMIZE_ALLOCATED_MEMORY`; pgrx’s default debug build introduced approximately 35% memory-randomization overhead that skewed measurements. The extension was compiled in release mode.

Methodology. Wall-clock time is measured inside PostgreSQL via `clock_timestamp()` and `EXPLAIN (ANALYZE)`. The reported results are averaged over five runs.

Test Datasets. We evaluated the extension on three datasets chosen to represent different JSON-processing scenarios.

Synthetic single-document dataset. The first dataset consists of synthetic JSON files ranging from 4 MB to 90 MB in size. Each file contains a single top-level object with a large `records` array, schema of which is shown in Listing 1. The dataset was generated programmatically in Python by repeatedly creating records with random values. Its purpose is to measure raw JsonPath execution performance on individual large documents, where per-query IO overhead is negligible.

Listing 1: Schema of generated single file JSONs

```
1 {
2   "records": [
3     {
4       "id": int,
5       "name": string,
6       "active": bool,
7       "email": string,
8       "phone": string,
9       "tags": [string],
10      "address": {
11        "city": string,
12        "zip": string,
13        "street": string
14      },
15      "scores": [int]
16    }
17  ]
18 }
```

D3 dataset. The second dataset is the DBLP Discovery Dataset (D3), a real-world collection of scholarly metadata occupying approximately 4 GB in `.jsonl` format. After import into PostgreSQL, each JSON document occupies roughly 1.7 KB. This dataset represents workloads consisting of millions of relatively small documents and allows us to compare performance on realistic data.

Synthetic JSONL dataset. The third dataset is a synthetic 10 GB `.jsonl` collection generated specifically for evaluating index-assisted execution. Its schema is shown in Listing 2. Each document contains several large arrays and nested objects, resulting in an average compressed size of approximately 396 KB inside PostgreSQL (and around 1MB outside of it). Additionally, the `hobby` field is present in only about 10% of documents, making it suitable for testing selective GIN-indexed queries.

Listing 2: Schema of generated JSONs in `.jsonl` dataset

```

1 {
2   "id": int,
3   "name1": string,
4   "active1": bool,
5   "email1": string,
6   "phone1": string,
7   "tags1": [string x 15000],
8
9   "address1": {
10    "city1": string,
11    "zip1": string,
12    "street1": string
13  },
14
15  "nested1": {
16    "nested2": {
17      "countries": [string x 30000]
18    }
19  },
20
21  "scores1": [int x 200--3000],
22
23  "name2": string,
24  "active2": bool,
25  "email2": string,
26  "phone2": string,
27  "tags2": [string x 15000],
28
29  "address2": {
30    "city2": string,
31    "zip2": string,
32    "street2": string
33  },
34
35  "scores2": [int x 200--3000],
36  "cities": [string x 30000]
37
38  "hobby": [string x 3-5] // only in around 10% of all generated jsons
39 }

```

5 Results

We first present results for the synthetic single-document JSON dataset, followed by the D3 dataset, and finally the synthetic `.jsonl` dataset evaluated with the GIN index enabled.

5.1 Benchmarks on single file JSONs

The `rsonpath` extension operates directly on JSON documents stored as plain text, allowing it to process inputs of up to 1 GB. In contrast, PostgreSQL’s native `JSONPath` implementation requires documents to be stored as `jsonb`. Consequently, our synthetic JSON files had to be converted to `jsonb` before they could be queried using `JSONPath`. PostgreSQL was only able to perform this conversion for documents up to approximately 90 MB in size. **This demonstrates the first major advantage of `rsonpath` over `JsonPath`: the ability to process JSON documents that are up to ten times larger.**

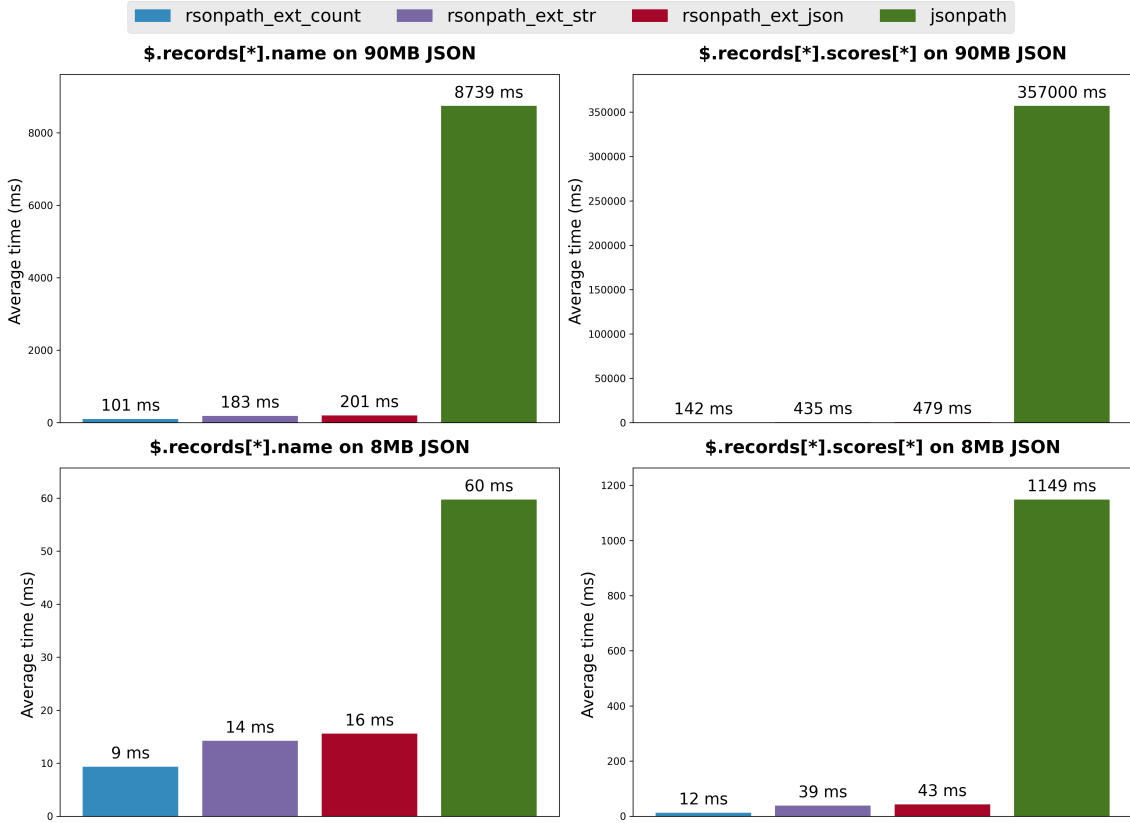


Figure 1: Performance comparison of `rsonpath` and PostgreSQL JSONPath on synthetic JSON documents of sizes 8 MB and 90 MB, evaluating simple (single-field) and recursive array (`[*]`) JSONPath queries.

Not only can the `rsonpath` extension process bigger JSONs than JSONPath, but it can also process them much faster. As we can see in Figure 1, for the 90MB JSON, `rsonpath_ext_json` takes 201 ms for `$.records[*].name` query and 479 ms for `$.records[*].score[*]` query, while JSONPath takes 8739 ms and 357000 ms. This means that **`rsonpath` is 43 and 745 times faster**, respectively. What is more, this benchmark shows that JsonPath doesn't handle queries of type `$.records[*].score[*]` (with `[*]` at the end) well.

For smaller, 8 MB JSON files, `rsonpath` remains faster than JsonPath, but the speedup is reduced to 3 and 26 times, respectively.

This suggests a hypothesis that the smaller the JSON gets, the lesser the advantage of `rsonpath` becomes.

5.2 Benchmarks on D3 dataset

In contrast to the benchmarks above, which use single large JSON files (8 MB and 90 MB), the D3 dataset consists of millions of much smaller JSON documents, each approximately 2 KB in size.

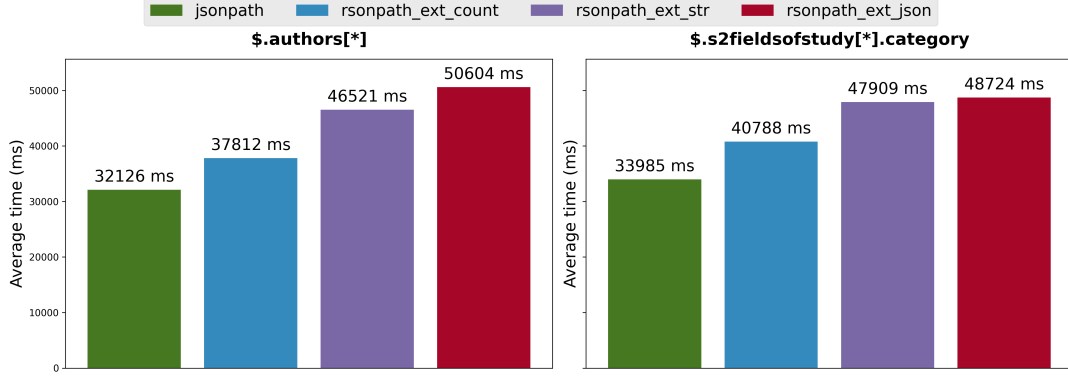


Figure 2: Performance comparison of rsonpath and PostgreSQL JSONPath on real world D3 dataset.

Figure 2 confirms the aforementioned hypothesis: as file size decreases, rsonpath’s advantage diminishes, eventually to the point where JSONPath becomes faster.

However, JSONPath for `$.authors[*]` and `$.s2fieldsofstudy[*].category` queries takes respectively 32126 ms and 33985 ms, while `rsonpath_ext_json` takes 50604 ms and 48724 ms, which means that **JSONPath is only around 1.5 times faster than rsonpath** in this case. So it’s not that huge of a difference, compared to the results we presented earlier.

There might be a few reasons for rsonpath’s loss in the above case:

1. Since rsonpath is an **external** extension, querying millions of rows requires PostgreSQL to pass data to the extension, which then forwards it to the rsonpath engine. This introduces significant overhead due to context switching and argument passing. For a single large JSON file, this occurs only once and is therefore negligible; however, in this case, it becomes the dominant cost. JSONPath doesn’t have these issues since it is built directly into PostgreSQL’s engine.
2. Rsonpath operates on a text format, meaning we always need to read the file from the beginning to the end, while JSONPath can easily skip parts of JSON, thanks to the `jsonb` format. As we can see, this skipping works relatively fast for small JSONs.

A more in depth attempt at explaining this is presented in the Discussion section 6.

5.3 Benchmarks on generated synthetic .jsonl dataset with GIN index set up

We implemented a custom GIN index for rsonpath and benchmarked whether it speeds up the search for the hobby key in our dataset; the hobby key is present in only around 10% of the rows in the dataset.

Explanation of the legend:

1. `jsonpath` and `rsonpath_ext_count` - baseline results, without GIN index set on our table
2. `jsonpath_gin_filter_only` and `rsonpath_gin_filter_only` - we use the GIN index to quickly check if a JSON document contains the specified path, we do not extract or count the actual matches, we only return the documents that contain the path.
3. `jsonpath_gin` and `rsonpath_ext_count_gin` - the full pipeline: the GIN index filters documents in the **WHERE** clause, and for each surviving document the matching engine is invoked again in the **SELECT** to produce the actual count. Concretely, for our extension this is:

```
SELECT sum(rsonpath_ext_count('$.hobby[*]', data::text))
FROM bench_jsonb
WHERE data @@@ '$.hobby[*]';
```

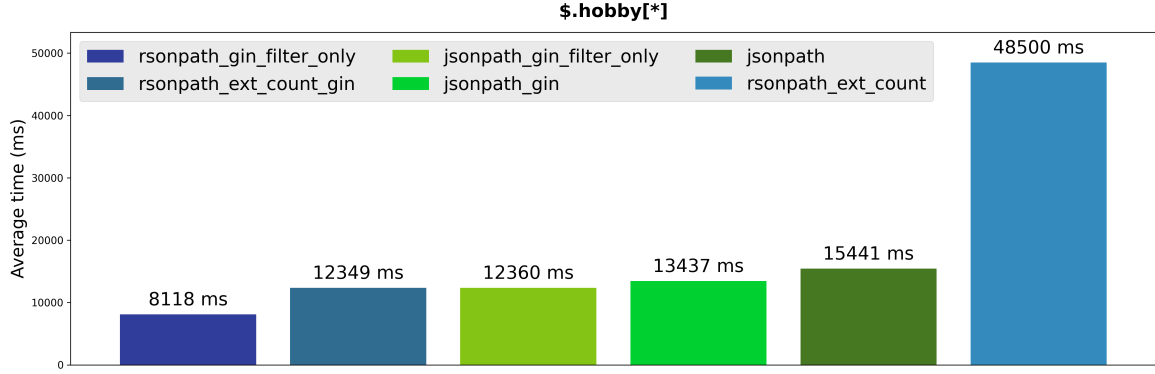


Figure 3: Performance comparison of rsonpath and PostgreSQL JSONPath on synthetic JSONL dataset with only around 10% of rows having `hobby` key; evaluating simple `$.hobby[*]` query.

As shown in Figure 3, even though the JSON documents in our generated JSONL dataset are larger than those in the D3 dataset (395 KB vs 2 KB), rsonpath still performs worse (48500 ms) than JSONPath (15441 ms), being approximately 3 times slower on `$.hobby[*]` query.

However, after setting up the GIN index for both rsonpath and JSONPath we can observe the substantial boost to rsonpath performance: **rsonpath_ext_count_gin is almost 4 times faster** and takes only 12349 ms instead of 48500 ms that we got from `rsonpath_ext_count`.

What is more, `rsonpath_ext_count_gin` achieves a similar execution time to `jsonpath_gin`, while `rsonpath_gin_filter_only` outperforms `jsonpath_gin_filter_only`. One possible explanation is that our custom GIN index performs only a simple key-based filtering step, whereas PostgreSQL’s native JSONPath GIN index may perform additional processing during index evaluation.

6 Discussion

6.1 Where does the time go?

The rsonpath CLI completes `$.records[*].scores[*]` on a 900 MB document in about 3 seconds; the same query inside PostgreSQL takes about 9 seconds. To understand the gap we profiled a representative PostgreSQL backend with `perf` during one such query. Figure 4 shows the resulting flame graph. The rsonpath engine occupies roughly 10% of CPU time. The dominant cost is PostgreSQL’s TOAST decompression (`pglz_decompress`, around 33%) and associated memory copying. The remainder is split between tuple construction and memory allocation overhead.

An auxiliary experiment in which TOAST compression was disabled (column storage set to `EXTERNAL`, rows rewritten) made queries *slower*, not faster: the increased I/O exceeded the saved CPU. PostgreSQL’s defaults are well-tuned for this workload.

6.2 Parallelism

Marking the extension functions `parallel_safe` lets the planner schedule the work across worker processes. On the many-rows dataset we observed approximately a 1.9× speedup using 7 workers on a 4-core CPU. The limited gain is consistent with the diagnosis above: the workload is partly I/O-bound, and hyperthreading provides only 20–30% extra throughput on top of physical cores.

6.3 GIN: when does it help, and what are its limits?

GIN is profitable when a query’s required-key set is selective: a small fraction of rows contain the keys mentioned in the JSONPath. For broad recursive queries (`$.*)` GIN cannot prune candidates and the index lookup degrades to a sequential scan with recheck overhead. For narrow queries, GIN can eliminate the majority of rows before rsonpath is invoked.

Several structural limits of our GIN integration are worth flagging for anyone continuing this work. `@@@` returns `bool` because GIN’s interface requires it to, so the operator can only filter – it cannot

References

- [1] M. Gienieczko, C. Paperman, *rsonpath: SIMD-accelerated JSONPath query engine*. <https://github.com/rsonquery/rsonpath>.
- [2] PgCentral Foundation, *pgrx: Build PostgreSQL extensions with Rust*. <https://github.com/pgcentralfoundation/pgrx>.
- [3] J. P. Wahle, T. Ruas, S. Mohammad, B. Gipp, *D3: A Massive Dataset of Scholarly Metadata for Analyzing the State of Computer Science Research*. LREC 2022. <https://github.com/jpwahle/lrec22-d3-dataset>.
- [4] Rsonpath extension code repository: https://github.com/KrzysztofLembryk/rsonpath_integration_with_postgres